Plan de pruebas de la aplicación Bicis

Historia de usuario: Detectar la ciudad automáticamente según mi ubicación (ID 532319).

Yo, como visitante temporal, quiero que la aplicación detecte la ciudad en la que estoy a partir de mi ubicación de manera que pueda ver sus estaciones nada más llegar, sin tener que buscar la ciudad en la lista.

Los niveles de prueba que vamos a usar son estos:

- Pruebas de aceptación. Son los criterios que acordamos con el Product Owner. Se hacen a mano en la Product Review.
- Pruebas de interfaz. Hay una por cada criterio. Las que se pueden automatizar están hechas con Espresso. Para poner la ubicación del móvil usamos la clase MockLocationManager que nos pasó la profesora, y para el caso en el que no hay ubicación, un proveedor de ubicación falso metido con Hilt.
- Pruebas unitarias. Se hacen a los métodos nuevos o que hemos cambiado en NetworkLocator y MainPresenter, con caja negra (partición equivalente y AVL) y JUnit. En el presenter, la vista, el repositorio y la ubicación son mocks de Mockito.

Abajo están los casos de prueba de cada nivel.

CRITERIOS DE ACEPTACIÓN

Prueba 1. Caso de éxito

Contexto: el usuario ha dado permiso de ubicación, tiene la ubicación del móvil activada y está a 20 km o menos de una ciudad que tiene red de bicis (por ejemplo, en Santander).

1. El usuario abre la app.
2. La app carga las redes de bicis.
3. La app saca la ubicación del móvil.
4. La app busca la red más cercana al usuario (ver NOTA).
5. La app abre directamente la lista de estaciones de esa red, sin que el usuario tenga que buscarla.
6. Se verifica que sale en pantalla el nombre de la ciudad detectada (por ejemplo, "Santander").
7. Se verifica que las estaciones que salen son las de la red de esa ciudad (por ejemplo, TUeBICI).

NOTA: Para saber en qué ciudad está el usuario no usamos el nombre de la ciudad ni ningún servicio de fuera. Cada red trae la latitud y la longitud de su ciudad, así que la app calcula la distancia en línea recta del usuario a cada red y se queda con la más cercana, si está a 20 km o menos. Las redes sin coordenadas, o con coordenadas que no tienen sentido (latitud fuera de -90 a 90 o longitud fuera de -180 a 180), no se miran.

Prueba 2. Varias redes cerca del usuario

Contexto: hay más de una red a 20 km o menos del usuario (por ejemplo, dos redes en la misma ciudad).

1. El usuario abre la app.
2. La app carga las redes y saca la ubicación del móvil.

3. Se verifica que se abre la lista de estaciones de la red que está más cerca.
4. Se verifica que sale el nombre de la ciudad de esa red.

Prueba 3. Permiso de ubicación denegado

Contexto: el usuario no da permiso para usar su ubicación, o ya lo había denegado antes.

1. El usuario abre la app.
2. La app carga las redes de bicis.
3. La app pide el permiso de ubicación y el usuario lo deniega.
4. Se verifica que sale el mensaje "No has dado permiso para usar tu ubicación. Elige una ciudad de la lista".
5. Se verifica que sale la lista con todas las ciudades para que el usuario elija una a mano.

Prueba 4. No se puede obtener la ubicación

Contexto: el usuario ha dado permiso, pero no se consigue su ubicación (tiene la ubicación del móvil quitada, no hay señal o tarda demasiado).

1. El usuario abre la app.
2. La app carga las redes de bicis.
3. La app intenta sacar la ubicación del móvil y no puede.
4. Se verifica que sale el mensaje "No se ha podido obtener tu ubicación. Elige una ciudad de la lista".
5. Se verifica que sale la lista con todas las ciudades.

Prueba 5. No hay red de bicis en la ciudad del usuario

Contexto: la ubicación se saca bien, pero no hay ninguna red a 20 km o menos (por ejemplo, el usuario está en Reinosa).

1. El usuario abre la app.
2. La app carga las redes y saca la ubicación del móvil.
3. Se verifica que sale el mensaje "No hay ninguna red de bicis en tu ciudad. Elige una ciudad de la lista".
4. Se verifica que sale la lista con todas las ciudades.

Prueba 6. No se pueden cargar las redes

Contexto: no hay internet o el servicio no responde.

1. El usuario abre la app.
2. La app intenta cargar las redes y no puede.
3. Se verifica que sale el mensaje "Ha habido un error cargando las redes".
4. Se verifica que no se pide el permiso de ubicación ni se intenta detectar la ciudad, porque sin redes no se puede.
5. Se verifica que la app no se cierra.

PRUEBAS DE INTERFAZ

Hay una prueba de interfaz por cada criterio. Las automáticas se lanzan con Espresso con los datos del Anexo 1, y el permiso de ubicación se da con GrantPermissionRule. En UI.1 y UI.5 la ubicación se pone con MockLocationManager, que cambia el GPS del móvil por uno de prueba. En UI.4 se usa un proveedor de ubicación falso con Hilt que siempre falla, porque con MockLocationManager no se puede hacer que el GPS no conteste. UI.2, UI.3 y UI.6 no se pueden

automatizar (en la tabla pone por qué), así que las cubren las pruebas unitarias y la prueba de aceptación a mano.

Tabla 1. Pruebas de interfaz

ID	Criterio	Implementación y entrada	Resultado esperado
UI.1	Prueba 1. Caso de éxito	DetectarCiudadExitoUITest. MockLocationManager en Santander (43.4623057, -3.8099803)	Se abren las estaciones de TUeBICI y sale "Santander"
UI.2	Prueba 2. Varias redes cerca	Manual: en los datos de prueba no hay dos redes en la misma ciudad. La lógica la cubre UMP.1f	Se abren las estaciones de la red más cercana
UI.3	Prueba 3. Permiso denegado	Manual: el permiso se pide con un diálogo del sistema que Espresso no puede pulsar. La lógica la cubre UMP.1b	Mensaje de permiso denegado y lista con las 4 redes
UI.4	Prueba 4. No se puede obtener la ubicación	DetectarCiudadSinUbicacionUITest. Proveedor de ubicación falso que falla	Mensaje "No se ha podido obtener tu ubicación..." y lista con las 4 redes
UI.5	Prueba 5. No hay red en la ciudad	DetectarCiudadSinRedUITest. MockLocationManager en Reinosa (43.0018, -4.1376)	Mensaje "No hay ninguna red de bicis en tu ciudad..." y lista con las 4 redes
UI.6	Prueba 6. No se pueden cargar las redes	Manual: en la versión de pruebas las redes se cargan de un fichero del propio proyecto. La lógica la cubre UMP.1d	Mensaje de error al cargar, no se pide el permiso y la app no se cierra

MÉTODOS A PROBAR

Estos son los métodos nuevos o que hemos cambiado en esta historia, por clase:

- **Clase NetworkLocator:** findNearestNetwork y distanceKm .
- **Clase MainPresenter:** init (ahora pide el permiso después de cargar las redes) y onLocationPermissionResult .
- **Clase DeviceLocationProvider:** requestCurrentLocation . Usa cosas de Android, así que se prueba con las de interfaz (UI.1 y UI.5).
- **Clase MainView:** requestLocationPermission, showLocationPermissionDenied, showLocationUnavailable, showNoNetworkNearby y showNetworkStations . Son de la vista, así que se prueban con las de interfaz.

PRUEBAS UNITARIAS

Pruebas unitarias de la clase NetworkLocator

Para sacar los casos de cada método usamos clases de equivalencia y AVL, intentando cubrir todas las sentencias y todas las condiciones. Las redes de partida son las del Anexo 1 (con las

coordenadas redondeadas a 4 decimales) y el radio es de 20 km si no se dice otra cosa. Están en NetworkLocatorTest.

- Método findNearestNetwork(networks, lat, lon, maxKm)

Tabla 2. Casos de prueba del método findNearestNetwork

Identificador	Entrada	Valor esperado
UNL.1a	(43.4600, -3.8000) (Usuario en Santander)	TUeBICI
UNL.1b	(40.4618, -3.7038) (A unos 5 km de Madrid)	BiciMAD
UNL.1c	(48.8566, 2.3522) (París, lejos de todas las redes)	null
UNL.1d	(40.4550, -3.7038) con otra red en (40.4500, -3.7038) (Dos redes en la misma ciudad)	La otra red (la más cercana)
UNL.1e	(43.4623, -3.8099) con redes sin location, con coordenadas null o fuera de rango, y una red null (Datos mal)	TUeBICI
UNL.1f	Lista vacía	null
UNL.1g	Lista null	null
UNL.1h	(43.3000, -3.8099), maxKm = distancia a TUeBICI y maxKm - 0,01 (Valor límite del radio)	TUeBICI / null

- Método distanceKm(lat1, lon1, lat2, lon2)

Identificador	Entrada	Valor esperado
UNL.2a	(43.4623, -3.8099), (40.4168, -3.7038) (Santander - Madrid)	338,8 km (± 1 km)
UNL.2b	(40.4168, -3.7038), (40.4168, -3.7038) (Mismo punto)	0 km

Pruebas unitarias del presenter MainPresenter

Aquí el presenter se prueba solo. La vista (IMainContract.View), el repositorio de redes (INetworksRepository) y la ubicación (ILocationProvider) son mocks de Mockito, y lo que se mira es a qué métodos de la vista llama el presenter en cada caso. Las redes de partida son TUeBICI y BiciMAD. Están en MainPresenterTest.

- Métodos init y onLocationPermissionResult

Tabla 3. Casos de prueba del presenter

Identificador	Entrada	Valor esperado
UMP.1a	Permiso concedido, ubicación (43.4623, -3.8099) (Santander)	showNetworkStations(TUeBICI) y ningún mensaje de error
UMP.1b	Permiso denegado	showLocationPermissionDenied() y no se pide la ubicación
UMP.1c	Permiso concedido, falla la ubicación	showLocationUnavailable()
UMP.1d	Falla la carga de redes	showLoadError() y no se pide el permiso
UMP.1e	Permiso concedido, ubicación (43.0018, -4.1376) (Reinosa, sin red a 20 km o menos)	showNoNetworkNearby()
UMP.1f	Otra red en (43.4500, -3.8099), ubicación (43.4520, -3.8099) (Dos redes cerca)	showNetworkStations de la red más cercana

REPORTE FINAL

Las pruebas las ejecuté el 23 de octubre de 2026 con la versión 00.01.00 de la app, en el emulador (API 29). Las automáticas también pasaron en GitHub Actions al meter la historia en develop. El plan y los tests los he hecho yo (Diego Alonso), menos la primera versión de UI.1, que la empezó Ramón Belmonte y luego la adapté yo. Las pruebas a mano (UI.2, UI.3, UI.6 y las de aceptación) las hice yo antes de la Product Review, y las de aceptación también se enseñaron delante del Product Owner.

Tabla 4. Resultado de las pruebas

Prueba	Dónde está	Autor	Resultado
UI.1	DetectarCiudadExitoUITest	Ramón Belmonte y Diego Alonso	Pasa
UI.2	A mano	Diego Alonso	Pasa
UI.3	A mano	Diego Alonso	Pasa
UI.4	DetectarCiudadSinUbicacionUITest	Diego Alonso	Pasa
UI.5	DetectarCiudadSinRedUITest	Diego Alonso	Pasa
UI.6	A mano	Diego Alonso	Pasa
UNL.1a a UNL.2b	NetworkLocatorTest (10 tests)	Diego Alonso	Pasan todas
UMP.1a a UMP.1f	MainPresenterTest (6 tests)	Diego Alonso	Pasan todas
Pruebas 1 a 6	A mano antes y durante la Product Review	Diego Alonso	Pasan

Fallos que encontramos gracias a las pruebas:

- **UI.1 y UI.5:** la ubicación de MockLocationManager no llegaba a la app. La app pedía la ubicación a otro proveedor y se quedaba con la del emulador. Cambiamos DeviceLocationProvider para que la pida al GPS (y si la última ubicación conocida tiene menos de 2 minutos, usa esa).
- **UI.4 y UI.5:** solas pasaban, pero lanzando todos los tests juntos a veces fallaban. Los mensajes son Toast y se quedaban en cola los de otros tests. Ahora se reintenta la comprobación hasta 30 segundos, y al resto de tests de interfaz les pusimos una ubicación falsa que no contesta, para que no saquen mensajes.
- **UI.4 y UI.5:** al meter Bici Norte en los datos de prueba (historia 532377) fallaban porque esperaban 3 redes en la lista. Se cambiaron a 4.
- **UI.1:** con la tarea 977946 la app abre directamente las estaciones de la red detectada, así que cambiamos el test para que mire que se abre TUeBICI y que sale "Santander".

ANEXO 1. DATOS DE PRUEBA

Los datos con los que se lanza la app en modo debug (res/raw/networks.json) y las ubicaciones que se simulan en las pruebas son los de las Tablas 5 y 6.

Tabla 5. Redes de bicis de prueba

Red	Ciudad	Latitud	Longitud
Bicing	Barcelona, ES	41.3850639	2.1734035
BiciMAD	Madrid, ES	40.4168	-3.7038
TUeBICI	Santander, ES	43.4623057	-3.8099803
Bici Norte	Bilbao, ES	43.263	- (no tiene)

Bici Norte la metieron en la historia 532377. Como no tiene longitud, no cuenta al buscar la red más cercana, pero sí sale en la lista de redes.

Tabla 6. Ubicaciones simuladas

Ubicación	Coordenadas	Se usa en
Santander	(43.4623057, -3.8099803)	UI.1, UMP.1a
Reinosa	(43.0018, -4.1376)	UI.5, UMP.1e
Error de ubicación	-	UI.4, UMP.1c